

1. PROJECT OVERVIEW

- **Project Title:** ComicCraft - AI Comic Story Creator using Gemini Models

2. INTRODUCTION TO CODE LAYOUT & ARCHITECTURE

In any production-grade software application, maintaining a clean, modular, and readable code layout is essential for long-term scalability and debugging. For ComicCraft, the project structure is explicitly organized following standard FastAPI MVC (Model-View-Controller) design patterns. This ensures complete separation of concerns between backend routing, AI service logic, layout construction, and frontend presentation.

3. DIRECTORY STRUCTURE ORGANIZATION

The project directory is structured logically to separate source code, dependencies, templates, and static assets:

- `app/`: Contains the core backend Python modules.
 - `main.py`: The entry point that initializes the FastAPI application and mounts static routers.
 - `routes.py`: Manages all HTTP endpoint routes (GET and POST) and orchestrates the AI workflow.
 - `gemini_flash.py` & `gemini_pro.py`: Encapsulate calls to Google Gemini AI models for structured outlines and creative narrative generation.
 - `image_generator.py`: Manages Hugging Face Stable Diffusion and Pollinations API calls with robust fallback mechanisms.
 - `layout_builder.py`: Combines textual elements and image paths into a structured dictionary.
 - `exporters.py`: Handles FPDF library calls to compile layouts into downloadable multi-page PDFs.

- `templates/`: Contains Jinja2 HTML templates (`index.html`, `comic_preview.html`, `export_success.html`) for dynamic frontend rendering.
- `static/`: Stores stylesheets, background images, generated comic panels (`static/panels/`), and exported PDF files (`static/exports/`).
- Root Level Files: `.env` for secure environment variable management, and `requirements.txt` for tracking package dependencies.

4. READABILITY AND CODING STANDARDS

- **PEP 8 Compliance:** All Python scripts adhere strictly to Python's PEP 8 style guidelines, featuring clear variable naming, consistent indentation, and proper spacing.
- **Modular Functions:** Monolithic scripts are avoided. Every file handles a single, well-defined responsibility (e.g., `image_generator.py` exclusively handles image fetching and exception management).
- **Comprehensive Docstrings and Comments:** Complex logical blocks, such as proxy handlers and AI prompt formatting, are heavily commented to make the codebase self-explanatory for external reviewers.

5. REUSABILITY AND SCALABILITY

- **Modular AI Functions:** The AI integration functions accept dynamic prompts as parameters, allowing them to be reused across different comic genres and styles without rewriting code.
- **Reusable Layout Components:** The layout builder creates abstract dictionaries representing panels, making it easy to scale the application from a 2-panel comic to a 10-panel comic without modifying the PDF exporter code.
- **Environment Configuration:** Sensitive credentials (`GEMINI_API_KEY`, `HF_API_KEY`) are decoupled from the source code using `python-dotenv`, ensuring the code can be reused across different deployment environments safely.

